# 01  CPython 3.15: code-first tour

*profiler / jit / free-threading / lazy imports / frozendict / utf-8*

```
python3.15 -m profiling.sampling attach $PID
PYTHON_JIT=1 python3.15 bench.py
if not sys._is_gil_enabled(): ...
lazy import json
cfg = frozendict(service="api", retries=3)

# theme:
# which 3.15 updates shift runtime decisions,
# not just syntax or release notes?
```

# 02  How to read this deck

*same topic, code-heavy format*

```
# every slide gives you:
# 1 change
# 1 runnable form
# 1 engineering rule

# default bias:
# less ceremony, more code

# rivals adopt these quietly:
# observability defaults
# startup wins
# safer stdlib ergonomics
```

# 03  The map

*20 slides, intentionally code-biased*

```
agenda = [
    "profiling package",
    "tachyon quickstart",
    "jit observability",
    "free-threading probes",
    "lazy imports",
    "frozendict / sentinel",
    "unpacking / utf-8 / quiet wins",
]

# deep: profiler + jit + free-threading
# fast sweep: import, data-model, encoding, stdlib speedups
```

# 04  PEP 799: one namespace, two tools

*profiling.tracing + profiling.sampling*

```
import profiling.tracing
import profiling.sampling

profiling.tracing.run("main()")
# exact calls / higher overhead

# python -m profiling.sampling attach 12345
# low-overhead / production-friendly

# cProfile stays as an alias
# profile is deprecated and scheduled for removal in 3.17
```

# 05 Tachyon quickstart

*official cli forms worth memorizing*

```
python -m profiling.sampling run script.py
python -m profiling.sampling run -m mypkg.cli arg1 arg2
python -m profiling.sampling run --flamegraph -o profile.html script.py
python -m profiling.sampling attach 12345
python -m profiling.sampling attach --live 12345
python -m profiling.sampling run --heatmap script.py

# attach rules:
# same minor version required
# FT <-> non-FT attach is not supported
```

# 06  Sampling vs tracing

*which profiler answers which question?*

```
python -m profiling.sampling run app.py
python -m profiling.sampling run --mode=gil -a app.py
python -m profiling.tracing -m mypkg.worker

# sampling -> hotspots
# tracing  -> exact call counts

# docs recommend sampling first for most work
# use tracing when fast calls must not be missed
```

# 07  profiling.tracing in code

*new name, familiar workflow*

```
import profiling.tracing
import pstats

with profiling.tracing.Profile() as pr:
    main()

pstats.Stats(pr).sort_stats("cumulative").print_stats(20)

# familiar cProfile-style workflow
# just in the new namespace
```

# 08  JIT: what code can observe

*availability, enablement, not guarantees*

```
import os
import sys

print(sys._jit.is_available())
print(sys._jit.is_enabled())
print(os.environ.get("PYTHON_JIT"))

# build-dependent feature
# process-level switch
# sys._jit is explicitly an implementation detail
```

# 09  JIT: the is_active() trap

*good for debugging the jit, bad for app logic*

```
for _ in range(BIG_NUMBER):
    if sys._jit.is_active():
        assert sys._jit.is_active()   # can fail by design

# docs: testing/debugging the jit only
# branching on is_active() is intentionally unsafe
```

# 10  JIT benchmark pattern

*measure curves, not one heroic number*

```
PYTHON_JIT=0 python3.15 -m pyperformance run
PYTHON_JIT=1 python3.15 -m pyperformance run

python3.15 -m timeit -s "from bench import f" "f()"

# docs on 2026-04-29:
# x86-64 Linux: ~6–7% geo mean
# AArch64 macOS: ~12–13%
# individual benchmarks still range from slowdowns to big wins
```

# 11 Free-threading: runtime probes

*ask the interpreter what build you really have*

```
import sys

print(sys._is_gil_enabled())
print(sys.flags.gil)
print(sys.abi_info.free_threaded)
print(sys.abi_info.pointer_bits)

# _is_gil_enabled() = runtime state
# abi_info.free_threaded = build capability
# 3.15 also adds sys.abi_info itself
```

# 12 Free-threading: same code, new ceil

*thread pools stop being a joke for some cpu work*

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(8) as ex:
    out = list(ex.map(cpu_bound, range(1000)))

# 3.15: mimalloc becomes default for raw allocations
# on free-threaded builds
# real wins still depend on extensions, sharing, workload shape
```

# 13  FT + Tachyon: profile contention

*cpu mode and gil mode answer different questions*

```
python -m profiling.sampling attach -a --mode=cpu 12345
python -m profiling.sampling attach -a --mode=gil 12345
python -m profiling.sampling attach --live 12345

# ask both:
# where is cpu time?
# who monopolizes python execution?
```

# 14  Lazy imports: syntax + restrictions

*explicit, readable, but not everywhere*

```
lazy import json
lazy from pathlib import Path

# invalid:
# lazy from pkg import *
# lazy from __future__ import annotations

# only at module scope
# not inside def / class / try
# import errors move to first use
```

# 15  Lazy imports at scale

*source-level opt-in and runtime control*

```
__lazy_modules__ = ["json", "pathlib"]

import sys
sys.set_lazy_imports("all")
sys.set_lazy_imports_filter(myapp_filter)

# filter(importing, imported, fromlist) decides laziness
# keep your app lazy and third-party deps eager
```

# 16 frozendict

*immutable mapping, hashable when its members are*

```
cfg = frozendict(service="api", retries=3)

scope = frozendict(answer=42)
print(hash(cfg))
print(eval("answer + 1", scope))

# not a dict subclass
# preserves insertion order
# equality ignores insertion order
```

# 17 sentinel

*finally: a first-class missing-value pattern*

```
MISSING = sentinel("MISSING")

def read(limit: int | MISSING = MISSING):
    if limit is MISSING:
        ...

# identity survives copy / pickle
# works in type expressions with |
# cleaner than object() sentinels
```

# 18  PEP 798: unpacking in comprehension

*less nesting, same intent*

```
[*L for L in lists]
{*s for s in sets}
{**d for d in dicts}
(*chunk for chunk in streams)

# direct alternative to nested comprehensions
# and chain / chain.from_iterable patterns
```

# 19  UTF-8 by default

*fewer locale surprises, but still be explicit*

```
open("report.md").read()
open("legacy.txt", encoding="locale")

python3.15 -X utf8=0 script.py
PYTHONUTF8=0 python3.15 script.py

# 3.15 default when encoding is omitted: UTF-8
# cross-version safe rule: pass encoding explicitly
```

# 20  Quiet wins + good questions

*small runtime changes that compound*

```
import base64
import csv

base64.b64encode(b"x" * 4096)
csv.Sniffer().sniff("a,b,c\n1,2,3\n")

# 3.15 docs:
# base64 encode ~2x, decode ~3x
# Base32 / Ascii85 / Base85 / Z85 much faster
# csv.Sniffer.sniff() up to 1.6x faster
# where will you test 3.15 first?
```